The Python Script (PythonScript) Agent

1 Introduction

The PythonScript agent is designed to allow for execution of Python scripts. Scripts will be run outside of the agent server process to protect the agent server process. Python scripts will have the ability to communicate to the agent server process via Adroit's OLE Automation component.

Future versions of the agent may allow for embedded running of python scripts.

2 Operation

Python must be installed on any agent server PC's requiring to run PythonScript agents.

The Adroit 11 installer will ensure that Python application as well as the pywin32 package is installed on the agent server computer. However, if Python has already been installed, the agent allows you to configure the path to python.exe.

On installing Adroit, you will have had the option of installing the Python environment, if you do not already have Python installed. Adroit will install Python to your C:\Program Files\Adroit Technologies\Python folder together with the pywin32 and adroit-py (Adroit OLE Python) packages. Python scripts are able to interact with the agent server using Adroit OLE Python package.

If you have Python already installed, you will need to

- install the PyWin32 package as follows:
  pip install pywin32
- install the Adroit OLE Python package as follows:
  pip install adroit-py

The Adroit OLE Python package (zip file) can also be download from [*to be decided*]. The Adroit OLE Python zip file contains:

- adroit_py-0.1.0-py3-none-any.whl - this file must be copied to your pc and installed as follows:

  pip install adroit_py-0.1.0-py3-none-any.whl

- myscript.py – a sample python script showing how to use the Adroit OLE package

The agent then allows you to specify the full path to the python script to be run. Python scripts should include the provided adroitole.py script for programmatic access to Adroit's OLE Automation component which will allow you to get (fetch) and set (poke) tag values.

The python script will run without a visible console window. Therefore …

- The use of print() statements in your scripts will never be visible
- **Do not use input() statements** in your python script otherwise the python.exe instance will remain in process space until manually terminated!!

The script can be configured to run synchronously or asynchronously. The script is always invoked on a separate thread to avoid holding up other agent server processes. In asynchronous mode the thread will terminate immediately after setting off the python process. In synchronous mode the thread will only terminate after the python process has terminated.

Once the thread is created to launch the script, a busy status bit will be set until the thread is terminated. Before terminating the thread will pulse the completed status bit to indicate completion.

The agent supports a list of trigger tags. Triggers tags may be individually enabled/disable. For each trigger tag the type of trigger may be configured viz any value change, rising edge (0->n) or falling edge (n->0).

Other than the ability to add trigger tags, this agent does not cater for scheduling of scripts

This agent does not support realtime catchup.

3 Scripting examples

3.1 Simple script

The following code shows the ability to execute a script with or without parameters

```
import sys
```

```
def HelloWorld():
    print("Hello World!")

def DoSomething(str):
    print(str)

# --- simply running a script file with no parameters
if __name__ == "__main__":
    if len(sys.argv) == 1:
        HelloWorld()
        sys.exit(0)

# --- running a script file with parameters DoSomething “Hello World”
    if len(sys.argv) == 3 and sys.argv[1] == " DoSomething ":
        DoSomething (sys.argv[2])
        sys.exit(0)

# --- if we get this far than the script was not called correctly, set agent’s error status to true
#     by exiting with a a non-zero code.  This will also show cause the agent’s info slot to show
#     “Process finished with error code: 1”
sys.exit(1)
```

3.2 Advanced script showing agent server interaction

The following code shows how to interact with Adroit

```
# Sample script demonstrating the use of AdroitOLE COM wrapper to interact with Adroit SCADA system.
# This script creates analog agents, sets alarms, scans tags, logs data, and performs various
operations.
# Revision: 1.0
# Date: 2026-01-22

from datetime import datetime, timedelta
from adroit import AdroitOLE
```

```
import random
import time

# instiantiate the AdroitOLE COM wrapper
adroit:AdroitOLE = AdroitOLE();

def main():

    try:
        # Connect to a remote Agent Server named SERVER01:ADROIT
        #adroit.Connect("SERVER01","ADROIT")

        # Generate a random number and set it to MyAnalog2.value
        num = random.randint(1, 1000)

        # Set the random value to MyAnalog2.value
        adroit.SetTag("MyAnalog2.value", num)

        # Fetch back the value to verify
        result = adroit.Fetch("MyAnalog2", "value")
        print(f"\r\nSet random value {num} to MyAnalog2.value, fetched back: {result}")

        input("\r\nPress any key to continue...")

        # Fetch multiple tags MyAnalog1.value and MyAnalog2.value
        results = adroit.FetchTags(["MyAnalog1.value","MyAnalog2.value"])
        print("\r\nFetched Tags values for MyAnalog1.value and MyAnalog2.value:", results)

        input("\r\nPress any key to continue...")

        # Fetch slot info for agent MyAnalog1
        results = adroit.GetSlotInfo("MyAnalog1")
        print("\r\nGet Slot Info for agent MyAnalog1:", results)

        input("\r\nPress any key to continue...")

        print("\r\nSetting multiple values (50) to MyAnalog2.value...Please wait.")
        # Setting multiple values (50) to MyAnalog2.value
        for i in range(50):
            # Generate a random number and set it to MyAnalog2.value
            num = random.randint(1, 1000)

            # Set the value to MyAnalog2.value
            adroit.SetTag("MyAnalog2.value", num)

            time.sleep(0.3)  # slight delay to simulate time between setting values

        # Fetch changes for MyAnalog2.value between two dates (last 5 seconds)
        results = adroit.FetchChanges("MyAnalog2.value", (datetime.now() -
timedelta(seconds=5)).strftime("%Y-%m-%d %H:%M:%S"), datetime.now().strftime("%Y/%m/%d %H:%M:%S"))
        print("\r\nFetch Changes for MyAnalog2.value during last 5 seconds:", results)

        input("\r\nPress any key to continue...")

        # Fetch values for MyAnalog2.value between two dates (last 5 seconds), limit to 5 results
        results = adroit.FetchValues("MyAnalog2.value", (datetime.now() -
timedelta(seconds=5)).strftime("%Y-%m-%d %H:%M:%S"), datetime.now().strftime("%Y/%m/%d %H:%M:%S"), 5)
        print("\r\nFetch Values for MyAnalog2.value during last 5 seconds, limit to 5 results:",
results)

    except Exception as e:
        # Catch and print any errors
        print(f"Error: {e}")

    input("\r\nScript completed. Press Enter to exit...")

if __name__ == "__main__":
    main()
```

Please note when running your Adroit enabled scripts from the command-line (cmd.exe) for testing purposes, you must be running the command-line window as administrator (elevated mode) otherwise you will not be able to communicate to the Adroit Agent Server which always runs in elevated mode.

4 Agent Slots

As of Rev

4.1 Agent Specific Slots

The following slots are available in the scheduler agent:

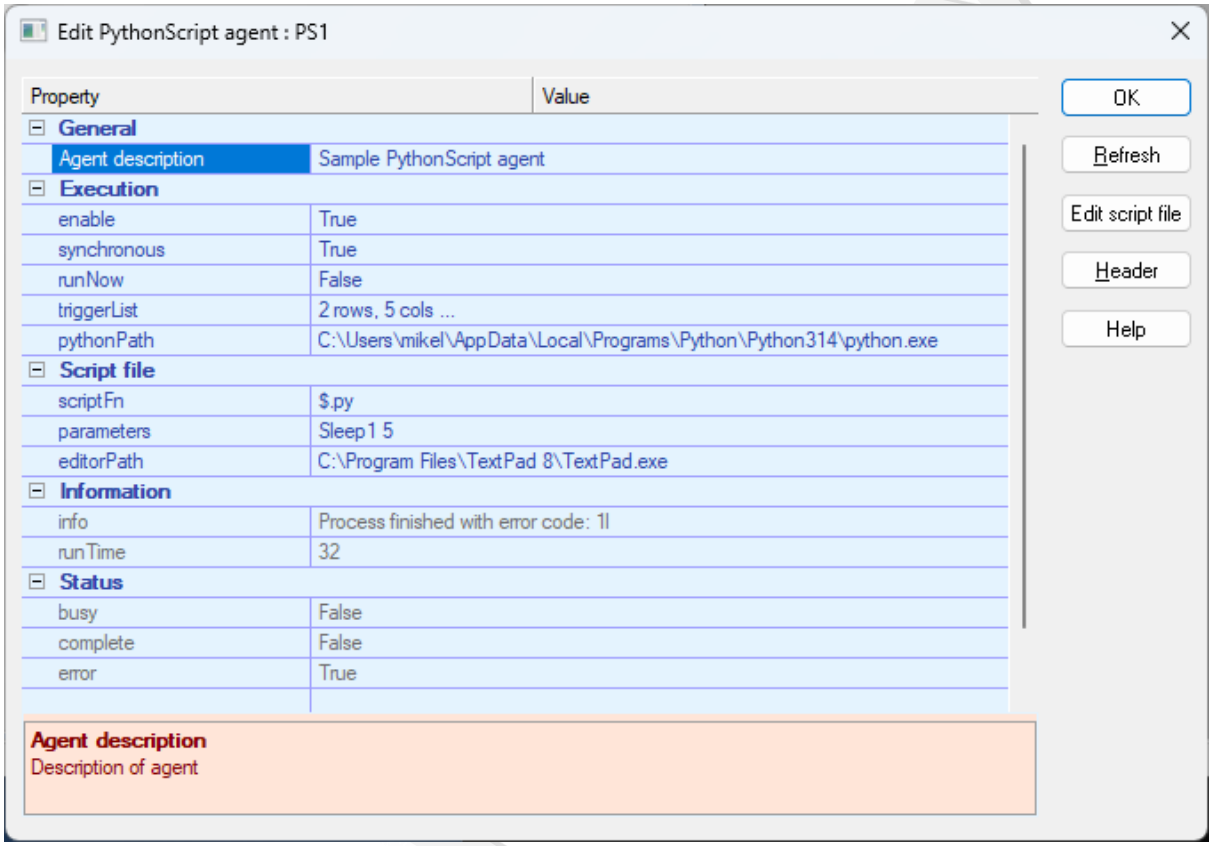
Slot name	Slot type	Slot description
autoReload	BOOLEAN	Monitor python script file timestamp and reload file if timestamp changes (for embedded mode only)
editorPath	STRING	Full file path to script code editor application – default `adrscriptedit.exe`
embedded	BOOLEAN	enable embedded mode – for future use only
enable	BOOLEAN	Enable agent
Info	STRING	Feedback information. If a script error occurs `sys.exit(`*nnn*`)` calls will set this slot value to show "Process finished with error code: *nnn*"
parameters	STRING	Optional parameters to be passed to script
pythonPath	STRING	Full path to Python.exe. Will default to `C:\Program Files\Adroit Technologies\Python\Python.exe` if it exists.
reloadNow	BOOLEAN	Reload script – only valid for embedded mode
runNow	BOOLEAN	Execute script now
runTime	BOOLEAN	Time taken for script to complete execution in ms
scriptFn	STRING	Filename of script to be launched. If scriptFn does not contain a full path, it will be searched for in the Adroit project data folder
synchronous	BOOLEAN	Run script synchronously – default 0
triggerList	LIST	List of trigger tags

4.2 Status slots

Status name	Status description
busy	Script has been launched and is running. In synchronous mode, bit will be turned off prior to setting complete status. In asynchronous mode, bit will be turned off when script has completed.
complete	Script has been launched (in synchronous mode) or completed (in asynchronous mode)
error	An error has occurred

5 Configuration

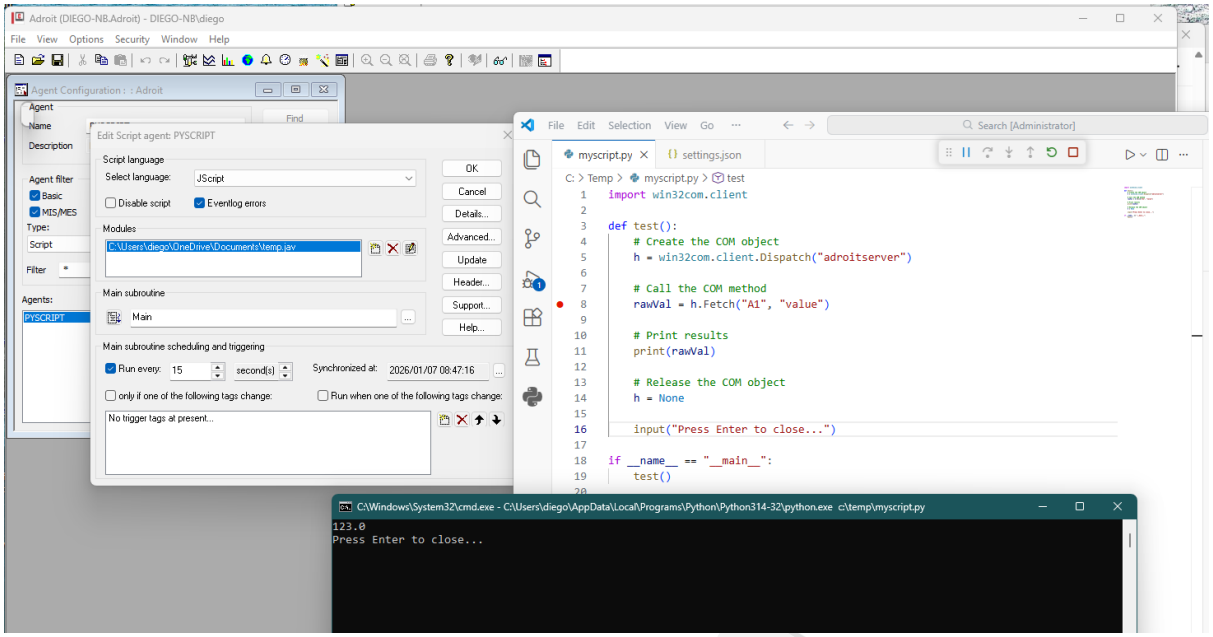
The configuration dialog will allow you to select and edit your python script file. By default, Adroit will attempt to open your code using adrscriptedit.exe (Scintilla host). Otherwise you can opt to use your own favourite script editor by specifying the full path to the editor executable.



The PythonScript agent dialog

6 Internal use only

From Diego



From Grok

https://grok.com/share/c2hhcmQtMi1jb3B5_f6c2eaa0-966a-43c9-812a-cf102b7f0958?rid=46069c34-c66c-4a77-9cc5-a1f886f48ee5

Please write mfc c++ sample code to integrate the execution of a python script file

Here are two common approaches to execute a Python script file from an MFC C++ application. The first is the **simplest** (using ShellExecute or system), ideal if you just need to run the script externally. The second uses **embedding the Python interpreter** via the Python/C API, which runs the script within your process (no console window, better integration, and allows capturing output or interacting with Python objects).

1. **1. Simple Method: Launch Python Interpreter Externally**

This runs python.exe script.py as a separate process. Add this code to a button handler in your MFC dialog (e.g., OnBnClickedButtonRunPython).

C++

void CYourDialog::OnBnClickedButtonRunPython()

```
{
    // Path to your Python script (adjust as needed)
    CString scriptPath = _T("C:\\path\\to\\your\\script.py");

    // Option A: Using ShellExecute (recommended - flexible, can hide window)
    ShellExecute(NULL, _T("open"), _T("python.exe"), scriptPath, NULL, SW_HIDE);  // SW_SHOW for
visible console

    // Option B: Using system() - simpler but always shows console briefly
    // CString cmd;
    // cmd.Format(_T("python.exe \"%s\""), scriptPath);
    // system(CT2A(cmd));

    AfxMessageBox(_T("Python script launched!"));
}
```

Notes:

- Ensure python.exe is in your system PATH, or provide the full path (e.g., C:\\Python312\\python.exe).
- Use SW_HIDE to run without a visible console window.
- This is quick and doesn't require linking Python libraries.

2. **2. Embedded Method: Run Python Script Inside Your Process (Using Python/C API)**

This embeds the Python interpreter. It's more powerful but requires setup.

Project Setup in Visual Studio:

- Include directories: Add Python include path (e.g., C:\Python312\include).
- Library directories: Add Python libs path (e.g., C:\Python312\libs).
- Linker > Input: Add python312.lib (match your Python version).
- Ensure matching architecture (32-bit or 64-bit).

Sample Code (in your dialog class, e.g., button handler):

C++

```
#include <Python.h>  // Must include this

void CYourDialog::OnBnClickedButtonRunPython()
{
    // Full path to the script file
    const char* scriptPath = "C:\\path\\to\\your\\script.py";

    // Initialize Python interpreter (call once, or guard if called multiple times)
    Py_Initialize();

    // Optional: Set sys.path to include script directory for imports
    wchar_t* wscriptPath = Py_DecodeLocale(scriptPath, NULL);
    if (wscriptPath) {
        wchar_t* dir = wcsdup(wscriptPath);  // Copy to modify
        wchar_t* lastSlash = wcsrchr(dir, L'\\');
        if (lastSlash) *lastSlash = L'\0';  // Truncate to directory

        PySys_SetPath(dir);  // Or append to existing path
        free(dir);
        Py_DecRef(wscriptPath);
    }

    // Open and run the script file
    FILE* fp = _Py_fopen(scriptPath, "r");
    if (fp) {
        int result = PyRun_SimpleFileEx(fp, scriptPath, 1);  // 1 = close file after
        if (result != 0) {
            AfxMessageBox(_T("Error executing Python script!"));
            // Optional: Print Python errors
            if (PyErr_Occurred()) PyErr_Print();
        } else {
            AfxMessageBox(_T("Python script executed successfully!"));
        }
    } else {
        AfxMessageBox(_T("Could not open Python script file!"));
    }

    // Finalize when done (or at app exit if embedding long-term)
    // Py_FinalizeEx();
}
```

Notes:

- Call Py_Initialize() once (e.g., in InitInstance() of your app) and Py_FinalizeEx() on exit for better performance.
- Use wide chars (wchar_t) on Windows for paths (as shown).
- To capture script output: Redirect sys.stdout/sys.stderr to custom objects or use PyRun_SimpleString for inline code.
- Error handling: Check PyErr_Occurred() and call PyErr_Print() for traceback.

- For advanced use (calling functions, passing args): Load as module with PyImport_ImportModule.

The embedded method is preferred for tight integration. Refer to Python docs: https://docs.python.org/3/extending/embedding.html for details.

If you need arguments passed to the script or output capture, let me know for extensions!

Please change the code showing how to call a function with parameters

Below is an updated **embedded Python** example for MFC C++ that shows how to:

1. Execute a Python script file.
2. Import it as a module.
3. Call a specific function inside that script **with parameters**.
4. Retrieve the return value (as an example, assuming the function returns a string or integer).

3. Example Python Script (myscript.py)

Save this as C:\path\to\myscript.py:

Python

```
def add_numbers(a: int, b: int) -> int:
    return a + b

def greet(name: str) -> str:
    return f"Hello, {name}!"

def process_data(x: float, y: float):
    print(f"Processing x={x}, y={y}")
    return x * y
```

4. MFC C++ Code – Calling a Python Function with Arguments

Add this to your MFC dialog class (e.g., in a button handler):

C++

```
#include <Python.h>
#include <atlbase.h>  // For CString conversions if needed

void CYourDialog::OnBnClickedButtonCallPythonFunc()
{
    // Initialize Python (do this once in your app, e.g., in InitInstance)
    if (!Py_IsInitialized())
        Py_Initialize();

    // Full path to the script
    const char* scriptPath = "C:\\path\\to\\myscript.py";
    const char* scriptDir  = "C:\\path\\to";  // Directory containing the script

    // Add script directory to sys.path so the module can be imported
    wchar_t* wDir = Py_DecodeLocale(scriptDir, NULL);
    PyObject* sysPath = PySys_GetObject("path");
    PyList_Append(sysPath, PyUnicode_FromWideChar(wDir, -1));
    Py_DecRef(wDir);

    // Extract module name from filename (without .py)
    CStringA moduleName = "myscript";  // or extract programmatically

    // Import the module
    PyObject* pModule = PyImport_ImportModule(moduleName);
    if (!pModule) {
        if (PyErr_Occurred()) PyErr_Print();
        AfxMessageBox(_T("Failed to import Python module!"));
        return;
    }
```

```
    // Get the function "add_numbers"
    PyObject* pFunc = PyObject_GetAttrString(pModule, "add_numbers");
    if (!pFunc || !PyCallable_Check(pFunc)) {
        AfxMessageBox(_T("Function 'add_numbers' not found or not callable!"));
        Py_XDECREF(pFunc);
        Py_DECREF(pModule);
        return;
    }

    // Prepare arguments: add_numbers(7, 5)
    PyObject* pArgs = PyTuple_New(2);
    PyTuple_SetItem(pArgs, 0, PyLong_FromLong(7));
    PyTuple_SetItem(pArgs, 1, PyLong_FromLong(5));

    // Call the function
    PyObject* pResult = PyObject_CallObject(pFunc, pArgs);
    Py_DECREF(pArgs);

    if (!pResult) {
        if (PyErr_Occurred()) PyErr_Print();
        AfxMessageBox(_T("Error calling Python function!"));
    } else {
        // Convert result to C++ type
        long result = PyLong_AsLong(pResult);
        CString msg;
        msg.Format(_T("Python function returned: %ld"), result);
        AfxMessageBox(msg);
    }

    // Cleanup
    Py_XDECREF(pResult);
    Py_DECREF(pFunc);
    Py_DECREF(pModule);

    // Optional: Finalize on app exit only
    // Py_FinalizeEx();
}
```

5. Calling a Function That Takes Strings and Returns a String

Example: calling greet("Alice")

C++

```
PyObject* pFuncGreet = PyObject_GetAttrString(pModule, "greet");
if (pFuncGreet && PyCallable_Check(pFuncGreet)) {
    PyObject* pArgs = PyTuple_New(1);
    PyTuple_SetItem(pArgs, 0, PyUnicode_FromString("Alice"));

    PyObject* pResult = PyObject_CallObject(pFuncGreet, pArgs);
    Py_DECREF(pArgs);

    if (pResult) {
        const char* str = PyUnicode_AsUTF8(pResult);
        CStringA resultStr(str ? str : "None");
        AfxMessageBox(CA2T(resultStr));
    }
    Py_XDECREF(pResult);
    Py_DECREF(pFuncGreet);
}
```

6. Key Points

- Always check PyErr_Occurred() or just call PyErr_Print() on failure.
- Use Py_DECREF/Py_XDECREF to avoid memory leaks.
- Initialize Python once (e.g., in CWinApp::InitInstance) and finalize on exit.
- For floating-point args: use PyFloat_FromDouble(value).

- For passing lists/dicts: build PyList_New or PyDict_New accordingly.

This approach gives you full control: pass parameters, get return values, and handle errors — all from your MFC C++ application. Let me know if you want to pass complex data (lists, dicts) or capture print() output!

How do I give the python scipt access to a COM object?

To give your embedded Python script access to a **COM object** (e.g., Excel, Outlook, or a custom MFC-exposed COM server), use the **pywin32** library (win32com.client). This works seamlessly in embedded Python, as long as you set up your project correctly.

7. **1. Prerequisites and Project Setup**

- Install **pywin32** in your Python environment (matching the embedded version): pip install pywin32.
- For embedded/distributed apps, include pywin32 DLLs (pythoncomXX.dll, pywintypesXX.dll) and run post-install if needed.
- Link your MFC project against the same Python libs.
- Call Py_Initialize() early (e.g., in InitInstance()).
- Importantly: Call pythoncom.CoInitialize() (or CoInitializeEx) before any COM usage in the thread running Python code. The main thread often initializes automatically via pywin32, but call it explicitly for safety.

C++

```
#include <pythoncom.h>  // From pywin32

// In your Python execution function or InitInstance
pythoncom::CoInitialize();  // Or pythoncom::CoInitializeEx(0, pythoncom::COINIT_APARTMENTTHREADED)
```

- Call pythoncom::CoUninitialize() on thread exit if needed.

8. **2. Python Script Accessing External COM Objects (Most Common)**

Your Python script can create and use standard Windows COM objects directly.

Example Python script (myscript.py):

Python

```
import win32com.client as win32

# Example: Control Excel
excel = win32.Dispatch("Excel.Application")
excel.Visible = True
wb = excel.Workbooks.Add()
ws = wb.ActiveSheet
ws.Cells(1, 1).Value = "Hello from embedded Python!"
ws.Cells(2, 1).Value = 42

print("Excel cell written!")

# Example: Outlook
# outlook = win32.Dispatch("Outlook.Application")
# mail = outlook.CreateItem(0)
# mail.Subject = "Test"
# mail.Body = "Hello"
# mail.Send()
```

No extra C++ code needed — just import the module and run as before.

9. **3. Passing a C++-Created COM Object (IDispatch*) to Python**

If you have an existing COM object in C++ (e.g., your MFC app exposes one via IDispatch), wrap it and inject it into Python globals.

Update your function (e.g., after importing the module):

C++

```
#include <Python.h>
#include <pythoncom.h>  // For COM init

void CYourDialog::PassComObjectToPython(IDispatch* pComDispatch, const char* pythonVarName)
{
   // Ensure COM initialized
   pythoncom::CoInitialize();

   // Create win32com Dispatch wrapper around raw IDispatch*
   PyObject* pWin32Module = PyImport_ImportModule("win32com.client");
   if (!pWin32Module) { PyErr_Print(); return; }

   PyObject* pDispatchFunc = PyObject_GetAttrString(pWin32Module, "Dispatch");
   Py_DECREF(pWin32Module);
   if (!pDispatchFunc || !PyCallable_Check(pDispatchFunc)) { Py_XDECREF(pDispatchFunc); return; }

   // Pass the IDispatch pointer as integer (void*)
   PyObject* pArgs = PyTuple_Pack(1, PyLong_FromVoidPtr(pComDispatch));
   PyObject* pWrappedCom = PyObject_CallObject(pDispatchFunc, pArgs);
   Py_DECREF(pArgs);
   Py_DECREF(pDispatchFunc);

   if (!pWrappedCom) { PyErr_Print(); return; }

   // Inject into module globals (or your script's locals)
   PyObject* pGlobals = PyEval_GetGlobals();  // Or PyModule_GetDict(pYourModule) if imported as module
   if (pGlobals) {
      PyDict_SetItemString(pGlobals, pythonVarName, pWrappedCom);
   }

   Py_DECREF(pWrappedCom);
}
```

Usage in C++ (assuming you have an IDispatch* myComObj):

C++

```
PassComObjectToPython(myComObj, "my_app_object");


// Then run your script — it can use my_app_object directly
```

In Python script:

```
Python

import win32com.client


# The object passed from C++

print(my_app_object.SomeProperty)

my_app_object.SomeMethod("test")
```

This mirrors VBScript's AddObject behavior.

10. 4. Full Example Integration (Button Handler)

Combine with previous embedding code:

```
C++

void CYourDialog::OnBnClickedRunScriptWithCom()
{
    Py_Initialize();
    pythoncom::CoInitialize();


    // Optional: Pass your custom COM object
    // IDispatch* pMyCom = ...; // Your existing COM instance
    // PassComObjectToPython(pMyCom, "app_com_obj");


    // Add script dir to path if needed...


    // Import and run module
    PyObject* pModule = PyImport_ImportModule("myscript");
    if (pModule) {
        // Optional: Call a function
        PyObject* pFunc = PyObject_GetAttrString(pModule, "main");
        if (pFunc && PyCallable_Check(pFunc)) {
            PyObject_CallObject(pFunc, nullptr);
        }
        Py_DECREF(pModule);
```

```
    }

    if (PyErr_Occurred()) PyErr_Print();

    // Py_Finalize() on app exit
}
```

11. **Tips & Troubleshooting**

- Use apartment-threaded model (COINIT_APARTMENTTHREADED) for most GUI/COM apps.
- For multithreaded Python execution, call CoInitialize per thread.
- Errors like "coinit not called" → Add explicit pythoncom.CoInitialize().
- Capture output/errors as shown previously.
- Test simple win32com.client.Dispatch("Excel.Application") first.

This gives your Python script full COM access, including external apps or your own C++/MFC COM objects. Let me know the specific COM object (e.g., Excel, custom) if you need a tailored example!

45 sources

Is there an alternative to COM for giving the python script access to C++ callback functions

Yes, there are several modern and more straightforward alternatives to **COM** for allowing an embedded Python script to call C++ functions (including callbacks). COM is Windows-specific, heavyweight, and overkill for this scenario. The preferred approaches use **Python binding libraries** that let you expose C++ functions, classes, and objects directly to Python as a module.

12. **Recommended Alternative: pybind11 (Best Choice in 2024+)**

pybind11 is a lightweight, header-only C++11+ library specifically designed for seamless bidirectional integration between C++ and Python. It's the most popular choice today for both extending (exposing C++ to Python) and embedding Python.

Why pybind11 over COM?

- No COM/IUnknown/IDispatch boilerplate.
- Modern C++ syntax, automatic type conversions (including STL containers).
- Excellent support for callbacks: Python functions can be passed to C++ and called later.
- Smaller binaries, faster compile times than older tools like Boost.Python.
- Full embedding support via pybind11/embed.h.

Setup in Your MFC Project

13. Download pybind11 (header-only): Include the include/ folder in your project (via submodule, vcpkg, or download).

14. Link against Python libs as before (python3xx.lib).
15. For embedding + exposing: Include <pybind11/embed.h>.

Example: Expose C++ Callbacks to Python

Assume you have a C++ class with a callback mechanism:

C++

```
#include <pybind11/pybind11.h>
#include <pybind11/embed.h>   // For embedding
#include <pybind11/functional.h>  // For std::function callbacks
#include <string>
#include <functional>

namespace py = pybind11;

class MyApp {
public:
   using Callback = std::function<void(const std::string&)>;

   void setCallback(Callback cb) {
      m_callback = cb;
   }

   void triggerEvent(const std::string& msg) {
      if (m_callback) m_callback(msg);
   }

private:
   Callback m_callback;
};

// Expose to Python as a built-in module
PYBIND11_EMBEDDED_MODULE(app_module, m) {
   py::class_<MyApp>(m, "MyApp")
      .def(py::init<>())
      .def("set_callback", &MyApp::setCallback)
```

```
        .def("trigger_event", &MyApp::triggerEvent);

    // Expose a global instance for easy access from scripts
    m.attr("app") = py::cast(new MyApp());  // Or use a singleton
}
```

Running the Script (in Your MFC Code)

```
C++
void CYourDialog::RunPythonScriptWithCallbacks()
{
    py::scoped_interpreter guard{};  // Starts interpreter (call once)

    // The module is automatically available when embedded
    py::exec(R"(
        def on_event(message):
            print("Python callback received:", message)
            # Do something in Python, e.g., update UI via other exposed funcs

        app.set_callback(on_event)
        print("Callback registered from Python!")
    )");

    // Now trigger from C++
    py::object app = py::module_::import("app_module").attr("app");
    app.attr("trigger_event")("Hello from C++!");

    // Or access your global instance directly
}
```

In Your Python Script File

```
Python
# myscript.py
def my_callback(msg):
    print("Event:", msg)
    # Call other exposed C++ funcs if needed
```

app.set_callback(my_callback) *# Register Python func as C++ callback*

Then load/run the file as before (py::eval_file("myscript.py")).

16. Other Alternatives

- **nanobind** → A faster successor to pybind11 (from the same author), even smaller/simpler for modern C++ (C++17+). Use if you need maximum performance.
- **Boost.Python** → Older but still works; similar to pybind11 but heavier (requires full Boost). Good if you're already using Boost.
- **Manual Python/C API** → Possible but verbose and error-prone (lots of PyObject* refcounting). Avoid unless you have very specific needs.
- **SWIG** → Generates bindings automatically from headers; good for large APIs but more setup and less "native" feel.

pybind11 is the cleanest, most maintainable way without the complexity of COM. It's widely used in projects like TensorFlow, PyTorch bindings, and many embedded scripting systems.

If you need a full MFC-integrated example (e.g., exposing MFC UI callbacks safely across threads), let me know!